

How To Use Loka — Unified Usage Guide

Version: 2.1 **Date:** 10 May 2026 **Purpose:** End-to-end practical guide for using the Loka framework series. Read this once, refer back occasionally.

The 30-Second Summary

- 1. Pick your version (`LOKA_INDEX.md` has the decision tree).
- 2. Open the **Lite** file for that version.
- 3. Paste it as the first message of your AI session (or as the system prompt).
- 4. Paste your task.
- 5. The AI follows the rules from there.

For long projects, also create the repo state files Loka asks for. The Full versions contain templates.

Choosing The Right Version (one more time)

Your Situation	Use
Writing prompts only	LokaV10
Planning architecture, no code yet	LokaV10
Auditing existing prompts/code	LokaV10
CLI/IDE coding agent (Claude Code, Cursor, Aider, Codex CLI, Gemini CLI)	LokaV11
Targeted scoped fix or surgical edit inside a repo	LokaV11 — Surgical Fix Mode
Long unsupervised run (1–3 hours, you check in periodically)	LokaV11 + LokaV12
Google AI Studio Build mode (browser, prompt-to-app)	LokaV13

Google Antigravity IDE (local install, agent-first)	LokaV13
Lovable, v0, Bolt, Replit Agent (visual prompt builders)	LokaV13
Exported from AI Studio / Antigravity to a real IDE	Switch to LokaV11
Don't know	LokaV11 Lite

Lite vs Full

Form	Length	Best For
Lite	1 page	Daily use. Paste as system prompt. Lower context cost.
Full	8–15 pages	Onboarding. Reference. Designing complex systems.

The Lite version is sufficient for almost all real work. The Full version is documentation that explains why each rule exists and gives templates.

Setup For A New Project

For any non-trivial project, do this once at the start:

1. Pick your version (V11 or V13 most likely).
2. Paste the Lite prompt at the start of a fresh AI session.
3. Ask the AI to create the initial repo state files based on your idea:
 - For V11: `SPEC.md` , `MEMORY.md` , `STACK.md` , `DO_NOT_TOUCH.md` , `SECURITY.md` , `VERIFY.md` (and `DESIGN.md` if UI, `EVALS.md` if AI)
 - For V13 (AI Studio Build / Antigravity IDE): `APP_ARCHITECTURE.md` , `DO_NOT_TOUCH.md` , `SECURITY_NOTES.md` , `UI_GUIDE.md` , `CHANGELOG_AI_STUDIO.md` , `HANDOFF.md` — plus Antigravity IDE will maintain `.gemini/antigravity/brain/` automatically
4. Review the files. Edit anything wrong. Lock SPEC.
5. Begin implementation.

Setup For An Existing Messy Project

1. Use **LokaV13 Rescue Mode** if you're in AI Studio Build mode or Antigravity IDE.
 2. Use **LokaV11 with Audit mode** if you're in a real IDE.
 3. Phase 1 is always damage assessment — no changes. Get a clear picture before fixing.
 4. Stabilise before refactoring. Document what works (even if ugly) before changing it.
 5. Fix one thing per prompt (V13) or per wave (V11). Verify after each.
-

Daily Use Patterns

Quick Bug Fix

```
Use LokaV10 Lite. Mode E (Fix).  
Bug: [describe]  
Symptoms: [observed]  
Where I think it is: [file/area]  
What I've tried: [list]
```

Surgical Edit in Existing Repo

```
Use LokaV11 Lite. Mode: Surgical Fix.  
Target change: [one sentence — what and why]  
Read MEMORY.md and STACK.md first.  
Declare files to touch and files NOT to touch before starting.  
TDD: write test → confirm fail → fix → confirm pass → regression check.
```

UI Tweak In AI Studio Build Mode or Antigravity IDE

```
Use LokaV13 Lite. Mode: UI Patch.  
  
Environment: [AI Studio Build / Antigravity IDE]  
Terminal policy (Antigravity IDE only): [Request Review / Auto]  
  
Task: [describe visual change]  
This is UI-only. Do not touch backend/DB/auth/firestore.rules.  
State files to update after: CHANGELOG_AI_STUDIO.md
```

Full-Stack Feature in Antigravity IDE

Use LokaV13 Lite. Mode: Feature Add.

Environment: Antigravity IDE

Terminal policy: Request Review

Shell sandbox: enabled

Goal: [feature]

Layers affected: [list]

Read APP_ARCHITECTURE.md and DO_NOT_TOUCH.md first.

Run dependency check before adding any new packages.

Pause at architectural checkpoints – do not self-approve.

Update CHANGELOG_AI_STUDIO.md and APP_ARCHITECTURE.md after.

Implement A Feature With Claude Code

Use LokaV11 Lite.

Mode: Implementation Engineer.

Goal: [feature]

Read SPEC.md, MEMORY.md, DO_NOT_TOUCH.md first.

Smallest safe change. Update STATE.md. Provide evidence.

Long-Run Build Session

Use LokaV11 + LokaV12 Lite.

[paste the V12 Launch Prompt with your mission brief]

Begin with Pre-Flight Check. Wait for my confirmation.

Architecture Planning Before Building

Use LokaV10 Lite. Mode B (App Architect).

Goal: [SaaS idea]

Sensitive data: [type]

Constraints: [budget/time/skill]

Output: SPEC + threat model + implementation waves

State File Discipline

Loka's discipline depends on the state files. Without them, you have rules but no anchor.

Minimum State Files By Project Type

Project Type	Required Files
Quick prototype	SPEC.md, STATE.md
Real MVP	+ MEMORY.md, STACK.md, DO_NOT_TOUCH.md, VERIFY.md
Production-bound	+ SECURITY.md, DECISIONS.md
UI-heavy	+ DESIGN.md
AI features	+ EVALS.md
Long-run sessions	+ LONG_RUN.md, end-of-session reports
AI Studio Build mode	APP_ARCHITECTURE.md, DO_NOT_TOUCH.md, SECURITY_NOTES.md, UI_GUIDE.md, CHANGELOG_AI_STUDIO.md, HANDOFF.md
Antigravity IDE	Same as AI Studio Build + <code>.gemini/antigravity/brain/</code> (auto-managed)

Maintenance Rhythm

- **Every session start:** AI reads SPEC, MEMORY, STATE, DO_NOT_TOUCH
- **Every meaningful change:** STATE.md updated
- **Every architectural decision:** DECISIONS.md updated
- **When facts solidify:** MEMORY.md updated
- **When context is degrading:** HANDOFF.md written
- **Every prompt in AI Studio Build mode:** CHANGELOG_AI_STUDIO.md updated — this is the only durable session memory in the browser environment

When Loka Will Push Back

Loka is opinionated. The AI will sometimes refuse to do what you asked. Common cases:

--	--

You Asked	Loka Will Say
"Just add a quick auth bypass for testing"	Will require explicit "TEMPORARY HACK — DO NOT DEPLOY" comment + tracked removal task
"Production-ready, right?"	Will give a specific label and list what's missing
"Run the tests" (no shell access)	Will say it can't, give you the command instead
"Refactor while you're at it"	Will list suggestions, won't execute without explicit approval
"Just disable the linter"	Will refuse — that's a kill-switch trigger in V12
"Drop the table"	Will require per-action explicit approval
"Use whatever library is convenient"	Will check STACK.md first, run Dependency Verification if shell is available
"Set terminal to Always Proceed"	Will warn this is unsafe for real projects and recommend Request Review
"This version is the latest, right?" (no verification run)	Will qualify: "as of training knowledge — running verification now" (or flag if shell unavailable)

These are features, not bugs. Override them only when you've thought it through.

Common Mistakes

1. Loading multiple Loka versions at once

Don't paste V10 + V11 + V13 together. Pick one. Mixing them wastes context and confuses the model.

2. Skipping the Pre-Flight Check (V12)

The Pre-Flight is the kill-switch on bad sessions. Don't skip it because the task feels small.

3. Treating Lite as a downgrade

Lite is the production version. Use Full when teaching, onboarding, or designing.

4. Trusting preview or browser sub-agent visual check as full verification

(V13)

Preview proves it renders. Browser sub-agent confirms visual layout. Neither proves DB writes, auth, payments, or backend logic work. Always provide manual verification steps for non-visual logic.

5. Letting MEMORY.md drift

If MEMORY.md is wrong, the AI builds on wrong foundations. Curate it. Same applies to `.gemini/antigravity/brain/` in Antigravity IDE — review it periodically.

6. Pasting Loka into a chat that's already 100 messages deep

Start fresh. Loka is a foundation, not a mid-conversation correction.

7. Standing approval ("just always do that for me")

Per-action approval exists for a reason. Standing approval defeats the safety model. This is especially dangerous in Antigravity IDE with Auto or Always Proceed terminal policy.

8. Not running Dependency Verification before building (V11 / V13 Antigravity IDE)

Training knowledge about package versions goes stale. Always run `date + npm view <package> version` (or PyPI equivalent) at session start when shell is available. Never assert a version is "current" based on memory alone.

9. Using V13 after exporting to a real IDE

V13 governs visual builder environments. Once you export to a local repo and open it in a standard IDE, switch to V11 immediately. The two frameworks have different rules, different state files, and different verification models.

10. Setting Antigravity terminal policy to Always Proceed

There is a documented real-world case of an agent attempting `chmod -R 777` on a directory after hitting a Permission Denied error in Auto mode. Start at Request Review. Escalate carefully.

When To Skip Loka

Loka is overhead. Skip it for:

- Single-file scripts
- Throwaway experiments
- Prompt iteration on creative writing
- Personal toy projects with no users

Use it for:

- Anything with users besides yourself
 - Anything touching auth, payments, or PII
 - Anything you'd be embarrassed to leak
 - Anything where you need to come back next month and remember what you did
-

Composition Rules

V12 Loads On Top Of V11

For long-run sessions, paste both. V11 first (or its Lite), then V12.

V13 Stands Alone

Don't combine V13 with V11 simultaneously. V13 is the visual builder specialisation. If you're using AI Studio Build mode or Antigravity IDE and then export to a real repo, switch versions at that boundary — don't run both.

V10 Is The Foundation Of All

V11 inherits V10's principles. V12 inherits V11's. V13 takes the core disciplines and adapts them for the visual builder environment with terminal policy governance and browser sub-agent verification.

You don't need to load V10 separately if you're loading V11.

Updating Your State Files

State files drift if you don't maintain them. Set a rule:

- **End of every work session:** STATE.md updated

- **End of every week:** MEMORY.md and DECISIONS.md reviewed
- **End of every month:** SPEC.md re-read; if reality has drifted, update SPEC and lock
- **Before any production action:** SECURITY.md re-read
- **Every Antigravity IDE session start:** re-read `.gemini/antigravity/brain/` and verify it reflects current project reality

Stale state files are worse than missing ones because they actively mislead.

When Things Go Wrong

If the AI is doing something unexpected:

1. **Stop.** Don't keep prompting harder.
2. **Check what state files exist.** Are they accurate?
3. **Re-read DO_NOT_TOUCH.md.** Did the AI cross a boundary?
4. **Check the readiness label.** Did the AI claim "done" without evidence?
5. **Check git status** (V11) or CHANGELOG_AI_STUDIO.md (V13). What actually changed?
6. **Consider rolling back** to last checkpoint if V12 was running.
7. **Write a HANDOFF.md** if context is degraded.
8. **Start a fresh session** with the state files preserved.

Loka can't prevent every failure. It limits the blast radius.

Feedback Loop

Loka is a tool. It will get better with use.

- Note rules that fired correctly (kept you out of trouble)
- Note rules that fired incorrectly (false positives)
- Note rules that should have fired but didn't (failures)
- Update your local copy of Loka with project-specific rules in MEMORY.md

Project-specific overrides go in MEMORY.md. Universal improvements should be merged

back into the framework.

File Manifest Recap

LOKA_INDEX.md	← decision tree (v2.2)
LOKA_HOW_TO_USE.md	← this file (v2.1)
LOKA_FAILURES.md	← 20 anti-patterns (v1.1)
LokaV10.md	← Secure App Builder — Full
LokaV10_Lite.md	← 1-page
LokaV11.md	← Agentic Build OS — Full (v11.3)
LokaV11_Lite.md	← 1-page (v11.3)
LokaV12.md	← Supervised Long-Run — Full
LokaV12_Lite.md	← 1-page
LokaV13.md	← Agentic Visual Builder OS — Full (v13.2)
LokaV13_Lite.md	← 1-page (v13.2)

Final Honest Note

Loka does not make AI coding agents reliable. Current agents (May 2026) hallucinate, drift, and confidently claim work they didn't do. Loka reduces the surface area. It does not eliminate it.

Loka also does not make your app secure. It enforces secure-by-design discipline. Real production security requires testing, external review, monitoring, and incident response.

What Loka does well:

- Catches the common, repeated, expensive mistakes
- Gives you durable state when chat history fails
- Makes it harder for the agent to claim work it didn't do
- Gives you honest readiness labels instead of vibes
- Survives context window collapse

That's worth a lot. It's not magic.

END OF HOW TO USE v2.1